

MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)

Mini-Project Title: Mini-Project 3 — OCR Expense Tracker & Receipt Parser (Flutter & Dart)

Team / Student Name: Nguyễn Phan Nhật Quang

Submission Date: 08/10/2026

1. GENERAL INFORMATION & DELIVERABLE LINKS

- **Team Members:**
- Nguyễn Phan Nhật Quang — Student ID: 23IT.B176 — Role: Individual project (architecture, OCR & parser, database, UI/charts, testing) — Contribution: 100%
-  **Android APK releases:** [Latest release](#). No GitHub Release has been published yet; downloads become available after a successful `v*` tag workflow. The universal APK will be named `Expense-OCR-<tag>.apk`. APKs use debug signing for coursework sideloading, not Google Play.
-  **GitHub Repository:** <https://github.com/thiennn20/Expense-OCR.git>

Tech stack: Flutter 3.47.6 · Dart 3.13 · camera · google_mlkit_text_recognition (on-device Latin model) · sqflite · image · CustomPainter (no chart library).

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	Camera capture & image cropping	✓ Complete	Live <code>CameraPreview</code> with a flash cycle (off / auto / on / torch), tap-to-focus (<code>setFocusPoint</code> + <code>setExposurePoint</code>) with an animated focus ring, and a receipt framing overlay drawn by <code>CropOverlayPainter</code> . The still photo is cropped to the frame in a background isolate. The camera is released and re-acquired on app lifecycle changes. Gallery import and manual entry are fallbacks.
2	On-device text recognition	✓ Complete	<code>google_mlkit_text_recognition</code> (bundled Latin model, works offline, no cloud cost). ML Kit's time is measured with a <code>Stopwatch</code> and shown on the review screen. OCR runs on the cropped image, downscaled to ≤ 1600 px, to keep latency low.
3	Regex heuristic parser	✓ Complete	Reads totals in <code>150,000 VND</code> , <code>150.000 đ</code> , <code>1.250.000</code> , <code>150,000.00</code> and <code>45k</code> formats; dates in <code>DD/MM/YYYY</code> , <code>DD-MM-YY</code> , <code>YYYY-MM-DD</code> and <code>Ngày ... tháng ... năm ...</code> ; merchant names from 39 known brands or from the receipt header. Each line is scored (total keywords +8, currency mark +2, cash/change/VAT/discount lines -7). Diacritics are folded first, and O-for-0 OCR errors are repaired. 16 unit tests.
4	Interactive review screen	✓ Complete	Every field can be edited. Fields carry a confidence hint (high / medium / low, with low highlighted). All detected amounts appear as tap-to-use chips, and the raw OCR text is expandable. Amounts are formatted as you type (<code>150.000 đ</code>). Leaving an unsaved scan asks for confirmation and deletes its images.
5	Local database & categories	✓ Complete	<code>sqflite</code> table <code>expenses</code> with indexes on <code>date</code> and <code>category</code> . The five categories (Food, Study, Travel, Gear, Entertainment) are suggested automatically from brand and keyword matches. Supports create, edit, and delete with undo.
6	Receipt thumbnail caching	✓ Complete	The cropped JPEG and a 240 px thumbnail are saved in <code><app documents>/receipts/</code> . The database stores relative file names. List tiles decode at display size (<code>cacheWidth</code>).
7	CustomPainter charts	✓ Complete	Donut: sweeps in clockwise; tapping a slice (hit-tested by angle) thickens it and fades the others; legend is linked; Week / Month / All filter. Weekly bar chart: staggered bar growth, "nice" Y-axis steps, dashed grid, tooltip on tap, previous/next week navigation. No third-party chart package.
8	Extras	✓ Complete	History grouped by day with search and category filter, swipe-to-delete with undo, light/dark theme, 25 automated tests, <code>flutter analyze</code> with no issues.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

```

lib/
├── core/      AppScope (InheritedNotifier DI), theme, VND formatters, date helpers
├── models/    Expense, ExpenseCategory
├── data/      SqfliteExpenseRepository (+InMemory for tests), ReceiptImageStore
├── services/  OcrService, receipt_layout (row merge), ReceiptParser, receipt_scanner
├── state/     ExpenseStore (ChangeNotifier) + chart aggregates
└── ui/       screens (home, dashboard, history, camera, review), widgets (painters)

```

Data flow. Camera → `ReceiptImageStore.saveReceipt` (isolate: EXIF rotation → crop to frame → resize → thumbnail) → `OcrService` (ML Kit, then lines merged into rows) → `ReceiptParser` (pure Dart) → draft `Expense` → `ReviewScreen` → `ExpenseStore.add` → `SQLite`.

State management. One `ExpenseStore` (`ChangeNotifier`) is exposed through an `InheritedNotifier` (`AppScope`), so no extra package is needed. Writes go to the repository first, and the in-memory list changes only after the write succeeds. The chart aggregates (`totalsByCategory`, `dailyTotals`) are computed from that list.

Testability. The parser and layout logic never touch ML Kit types. They work on plain strings and boxes, so every heuristic has a unit test. The repository is an interface, and an in-memory version drives the widget tests and screenshot rendering.

Exception handling.

- Camera permission denial and device errors (`CameraException`) show an explanation with *Try again* and *Pick from gallery*.
- Focus and flash calls that a device does not support are ignored.
- If OCR fails after the image was saved, the image is deleted and an error snackbar is shown.
- An empty OCR result shows a “No text detected” card instead of empty fields.
- Database load errors are kept in the store, and save errors appear in a snackbar.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

Screenshots below were rendered from the real app widgets at phone size (1080×2340) by `screenshot_test/`, using sample data. On the camera screen the live feed is a simulated receipt photo fed through a fake `CameraPlatform`. Replace them with device captures from the demo video if required.

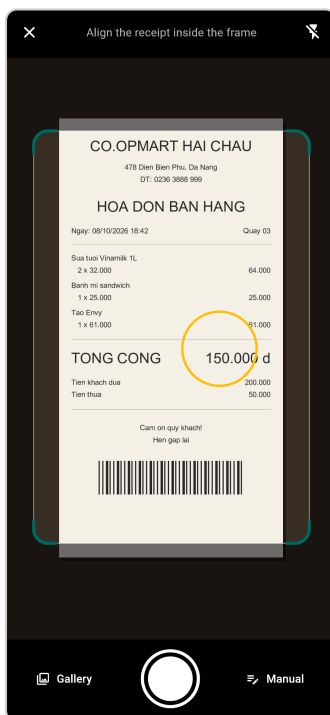


Fig. 1 – Capture. Framing overlay with corner brackets, tap-to-focus ring, flash toggle (top right), gallery / manual fallbacks.

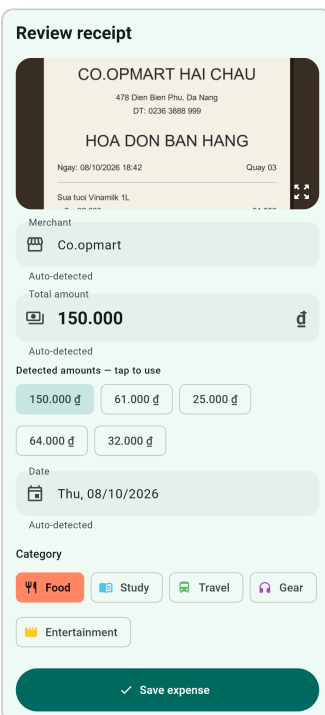


Fig. 2 – Review. Image cropped to the frame. Parser output: merchant *Co.opmart*, total *150.000 đ* (cash 200.000 and change 50.000 rejected), date 08/10/2026, category Food; other amounts as chips.

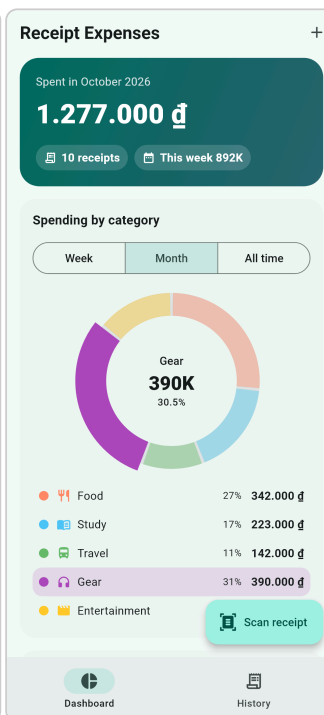


Fig. 3 – Donut chart. “Gear” slice selected by tap: it thickens, the others fade, and the centre and legend update.

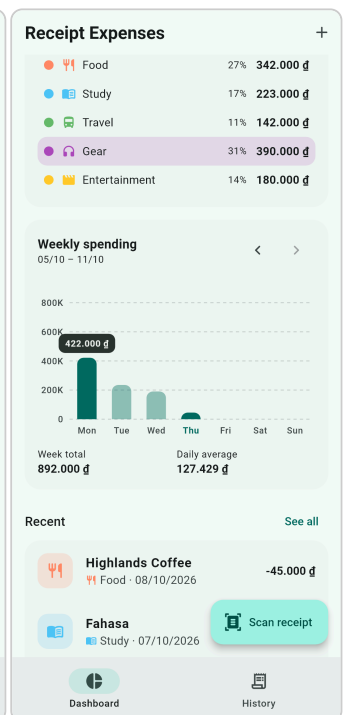


Fig. 4 – Weekly bar chart. Tooltip on the tapped bar, today (Thu) highlighted, “nice” axis steps, week navigation.

Verification results

- `flutter analyze` → *No issues found*. `flutter test` → **25/25 passed**: 16 parser tests (Vietnamese/English receipts, VAT-inclusive totals, label/value on separate rows, invalid and future dates, O-for-0 repair), plus chart geometry, store aggregates, and donut/bar tap interaction.
- `flutter build apk --release` → `app-release.apk` (85.3 MB universal). With `--split-per-abi`: arm64-v8a 31.9 MB, armeabi-v7a 25.6 MB. Most of the size is the bundled ML Kit model, which is what lets OCR work offline.

5. TECHNICAL CHALLENGES & RESOLUTIONS

1. ML Kit splits a receipt's columns into separate text blocks. ML Kit groups text by layout block. On a two-column receipt, "TỔNG CỘNG" sits in the left block and "150.000" in the right block, so they come back far apart, and a line-based regex cannot link the label to its value. *Resolution*: `mergeIntoRows` sorts all ML Kit lines by their vertical centre. A line joins the current row when its centre is within half a line height of the row, which tolerates slight camera tilt. Each row is then joined left to right, giving "TỔNG CỘNG 150.000" before parsing. When a label still ends up on its own row, the parser carries the keyword score over to the next row. Both cases are unit-tested.

2. Choosing the real total among many numbers. A receipt holds item prices, VAT, cash given (Tiền khách đưa 200.000) and change, as well as phone numbers, tax IDs and dates. Vietnamese receipts also use `.` and `,` as thousands separators, and OCR often reads 0 as o. *Resolution*: the parser runs in stages:

1. Fold diacritics and repair O-for-0 errors.
2. Remove dates, times and percentages; ignore phone and tax-ID lines.
3. Read number tokens by digit grouping, so `150.000` = `150,000` = 150000 and `150,000.00` keeps its decimal.
4. Score each candidate (total keyword +8, subtotal +2, currency mark +2, cash/change/discount/VAT -7, lower half of the receipt +1).

"đã gồm VAT" is not treated as a tax line. If no line scores high enough, the largest non-cash amount is used and marked low confidence for the user to check.

3. Crop mismatch between preview and photo; release-build issues. The preview stream and the still photo can have different aspect ratios, so cropping the photo with the on-screen frame fractions cut off part of the receipt. *Resolution*: the photo is first centre-cropped to the preview's aspect ratio (what the user actually saw), then the frame fractions are applied. This runs in a `compute` isolate, because decoding a 12 MP JPEG in Dart takes about a second and would freeze the UI. Two build issues came up for the release APK:

- On Windows, Gradle failed with "Could not close incremental caches" in `camera_android_camerax` and `image_picker_android`, because the project (drive E:) and the pub cache (drive C:) are on different drives. Setting `kotlin.incremental=false` in `android/gradle.properties` fixed it.
- The ML Kit plugin references the Chinese, Japanese, Korean and Devanagari recognisers only as `compileOnly` dependencies. With R8 minification turned on, `-dontwarn` rules for those packages were added to `proguard-rules.pro` so the shrinker does not stop on the missing classes.